

计组简答题

第二章

第五章

第六章

第八章

第二章

1. 为什么计算机内部采用二进制表示信息?既然计算机内部所有信息都用二进制表示,为什么还要学习十六进制表示?

答:主要有3个方面的原因:

(1)二进制系统只有两个基本符号:0和1。所以,它的基本符号少,易于用稳态电路实现。

(2)二进制的编码、记数、运算等规则简单。

(3)二进制中的0和1与逻辑命题的“真”和“假”的对应关系简单,为计算机中实现逻辑运算和程序中的逻辑判断提供了便利的条件,特别是能通过逻辑门电路方便地实现算术运算。

答:计算机内部在进行信息的存储、传送和运算时,都是以二进制形式来表示信息的。

但在屏幕上或书本上书写信息时,由于二进制信息位数多,阅读、记忆不方便,而十六进、八进制和二进制的对应关系简单,又便于阅读、记忆和书写,所以引入十六进制或八进制,使得人们在开发、调试程序和阅读机器内部代码时,能方便地用八进制或十六进制来等价表示二进制信息。

2. 常用的定点数编码方式有哪几种?通常它们各自用来表示什么信息?

常用的定点数编码方式有原码、反码、补码和移码。

原码用来表示整数和带符号数,反码用于计算机内部运算,补码用于计算机中的加减运算,移码用于浮点数表示。

3. 为什么现代计算机中大多用补码表示带符号整数?

现代计算机中大多使用补码表示带符号整数的原因有多个方面。

首先，补码的计算方式相对简单，可以通过对正数按位取反再加1得到其补码，这样可以简化计算机的硬件电路设计。

其次，补码可以统一处理正数和负数的加法运算，无需特殊处理符号位，简化了算术运算的实现。此外，补码唯一地表示了零，使得零的运算更加方便。

最后，补码还可以避免原码和反码的符号位溢出问题，提高了计算机的运算精度。

4. 在浮点数的基数和总位数一定的情况下,浮点数的表示范围和精度分别由什么决定?两者如何相互制约?

浮点数的表示范围和精度在给定基数和总位数的情况下分别由以下因素决定:

1.表示范围: 表示范围受到指数范围的影响。较大的指数范围允许表示更大的数,而较小的指数范围限制了表示范围。总位数也会影响表示范围,因为它限制了指数和尾数的位数。

2.精度: 精度取决于尾数的位数。更多的尾数位数允许表示更小的差异,因此提高了精度。但是,总位数的增加可能会减少指数的位数,从而降低表示范围。

表示范围和精度之间存在权衡关系。增加总位数通常会扩大表示范围,但可能降低精度,因为尾数位数减少了。相反,减少总位数可能提高精度,但会限制表示范围。

总的来说,浮点数的表示范围和精度是由基数、总位数、指数范围和尾数位数等因素共同决定的。在设计浮点数表示时,工程师需要仔细权衡这些因素以满足特定应用的需求。

5. 为什么要对浮点数进行规格化?有哪两种规格化操作?

为了使浮点数中能尽量多地表示有效位数,提高浮点数运算的精度,而且规格化形式具有唯一性。

规格化的操作有左规操作和右规操作。

6. 为什么计算机处理汉字时会涉及不同的编码(如输入码、内码、字模码)?说明这些编码中哪些用二进制编码,哪些不用二进制编码。为什么?

计算机处理汉字时涉及不同的编码是因为汉字的复杂性。汉字的数量庞大，无法直接用ASCII码等字符编码来表示。因此，为了能够在计算机中准确地表示和处理汉字，需要使用特定的编码方式。

在处理汉字时，输入码用于将用户输入的汉字转换为内码。输入码一般采用二进制编码，如GB2312、GBK、UTF-8等，这是因为计算机内部数据表示采用二进制形式，使用二进制编码可以方便地进行数据传输和处理。

内码是计算机内部使用的编码方式，用于表示和存储汉字。内码可以使用二进制编码，如Unicode，也可以使用其他编码方式，如汉字拼音、汉字的笔画等。内码的选择要考虑到存储空间和处理效率的平衡。

字模码是将汉字转换为点阵图形的编码方式。字模码一般不使用二进制编码，而是使用点阵编码，将汉字的每个点映射为一个位或字节。这是因为汉字的点阵图形比较复杂，使用二进制编码会导致存储空间和处理效率的问题。因此，字模码通常使用点阵编码，每个点的状态用一个位或字节表示。

第五章

1. CPU的基本组成和基本功能各是什么？

答:CPU中主要包含运算器、控制器以及一系列寄存器。

运算器主要功能是对数据进行加工处理,包括算术逻辑单元(ALU Arithmetic LogicUnit)、多路选择器、通用寄存器组及移位器组成。

控制器就是计算机系统的指挥中心,这一部件控制着计算机系统的各个功能部件,保证其能协同工作,自动执行计算机程序。

寄存器组是指CPU在运算或控制的过程中需要暂时存储信息的存储器集合。CPU中的寄存器主要可分为三类:处理寄存器、控制寄存器及主存接口寄存器，

2. 取指令部件的功能是什么？

取指令部件的功能是从存储器中读取指令，并将指令提供给控制器进行解释和执行。它通常包括以下几个步骤：

- 1、程序计数器（PC）的更新：首先，取指令部件需要获取下一条指令的地址。这个地址通常存储在程序计数器（PC）中。取指令部件会将PC中的地址发送给存储器。
- 2、指令的读取：根据PC提供的地址，取指令部件从存储器中读取指令内容，并将其存储在一个暂存器中，以供后续解释和执行。
- 3、PC的更新：一旦指令被读取，取指令部件会根据指令的长度和执行情况更新PC的值，使其指向下一条指令的地址。这样，在下一个周期中，可以从存储器中读取下一条指令。

3. 控制器的功能是什么？

控制器的功能是解释和执行指令，并控制整个CPU的各个部件协同工作。它通常包括以下几个主要功能：

- 1、指令解码：控制器对从取指令部件获取的指令进行解码，确定其具体操作和操作数[5]。通过解码，控制器能够理解指令的含义，并为后续的执行做出相应的安排。
- 2、控制信号的生成：根据指令的解码结果，控制器会产生针对不同部件的控制信号。这些信号包括操作数选择、ALU控制、数据传输和存储等操作的控制信号，用于指导各个部件的工作。
- 3、时序控制：控制器根据指令的执行过程和需要的时序，生成时钟信号和其他时序控制信号，以确保各个部件按照正确的顺序和时机进行工作。它协调整个CPU的操作，实现指令的有序执行。

4. 单周期处理器的CPI是多少？时钟周期如何确定？为什么单周期处理器的性能差？

单周期处理器中所有指令的指令周期为一个时钟周期，因此其 CPI（每条指令的时钟周期数）为 1。

单周期处理器的时钟周期由执行时间最长的那条指令决定。因为所有指令都要在一个时钟周期内完成，所以这个时钟周期必须能够容纳最复杂指令的全部执行过程，包括取指、译码、执行、访存和写回等操作。不同的硬件结构和指令集对时钟周期的确定也有影响。

由于时钟周期要满足最长指令的执行时间，导致时钟频率通常较低。相比之下，多周期处理器或采用流水线等技术的现代处理器，可以将指令的执行分解为多个阶段，每个阶段在一个较短的时钟周期内完成，从而能够设置更高的时钟频率，提高整体性能。

在单周期处理器中，指令是顺序执行的，同一时刻只能执行一条指令，硬件资源在大部分时间内可能处于空闲状态，无法充分发挥其并行处理能力。而多周期处理器或流水线处理器可以同时处理多条指令的不同阶段，提高了硬件资源的利用率，进而提升了处理器的吞吐量。

对于简单指令来说，其实际执行时间往往远小于时钟周期，但仍然需要等待整个时钟周期才能完成，这就造成了时间的浪费。特别是在处理大量简单指令的情况下，这种效率低下的问题会更加明显，导致处理器整体性能不佳。

5. 单周期方式下,在一个指令周期内某个部件能否被重复使用多次?为什么?

不能。

单周期方式的特点是每个指令的执行都在一个固定长度的时钟周期内完成。在这个周期内，各个部件按照固定的时序协同工作，完成指令的执行。

由于指令周期的长度是固定的，并且各个部件的操作也是按照预定的顺序依次进行的，一旦某个部件完成了其在当前指令周期内的任务，它就不能再次被使用，直到下一个指令周期开始。

6. 多周期处理器的设计思想是什么?每条指令的CPI是否相同?为什么在一个指令周期内某个部件可被重复使用?

设计思想：多周期处理器将一条指令的执行分解为多个阶段，每个阶段在不同的时钟周期内完成。

每条指令的CPI通常不同。

在多周期处理器中，指令的执行被分解成多个阶段，某个部件在完成一个阶段的任务后，在后续阶段可能再次被使用，因为不同阶段的操作可能会需要相同部件的参与。

7. 在控制器设计方法上,单周期处理器和多周期处理器的差别是什么?

指令执行周期：单周期处理器中，每条指令都在一个固定长度的时钟周期内完成。而多周期处理器将指令执行分解为多个阶段，不同指令可能需要不同数量的时钟周期。

部件使用效率：在单周期处理器里，部件在一个指令周期内不能被重复使用。但在多周期处理器中，部件在一个较长的指令执行过程中可被重复使用，因为指令执行被分成多个阶段。

灵活性：多周期处理器在控制器设计上更具灵活性，能适应不同复杂程度的指令。单周期处理器相对较僵化，所有指令执行时间相同。

硬件复杂度：多周期处理器的控制器设计通常更复杂，而单周期处理器相对简单。

8. 硬连线控制器和微程序控制器的特点各是什么？

硬连线控制器：

- ①硬连线控制器是一种基于组合逻辑电路实现的控制器。
- ②它的操作是通过硬连线的组合逻辑电路实现的具有固定的功能和指令集。
- ③对于每个指令，硬连线控制器的行为是提前定义好的，无法修改或扩展。
- ④硬连线控制器的设计和制造成本相对较低，运行速度快，但灵活性较低。

微程序控制器

- ①微程序控制器是一种基于存储器和微指令的控制器。
- ②它的操作是通过存储器中存储的微指令序列来实现的，每条微指令对应一个具体的操作。
- ③微程序控制器具有可编程性，可以修改和扩展微指令序列，从而改变控制器的行为。
- ④微程序控制器的设计和制造成本相对较高，运行速度较慢，但具有较高的灵活性和可编程能力。

9. 为什么CISC大多采用微程序控制器,RISC大多采用硬连线控制器？

- CISC（复杂指令集计算机）指令系统复杂，指令数量多且长度不固定。
- 微程序控制器具有较好的灵活性，便于实现复杂指令的控制逻辑。
- 能够通过修改微程序来增加新的指令或改进现有指令的功能。

- RISC（精简指令集计算机）指令系统简单，指令格式和长度固定。
- 硬连线控制器速度快，能满足 RISC 对指令执行速度的高要求。
- 由于指令简单，控制逻辑相对简单，适合用硬连线实现。

10. CPU检测内部异常和外部中断的方法有什么不同？

检测内部异常的方法：

- 内部异常通常是由 CPU 执行指令时产生的错误或特殊情况引起。
- CPU 会在指令执行过程中自动检测某些条件，如算术运算溢出、除数为零等。
- 这些检测通常是基于硬件电路和内部的状态标志来实现的。

检测外部中断的方法：

- 外部中断是由外部设备向 CPU 发送的请求信号。
- CPU 通过特定的中断引脚来接收这些信号。
- 当外部设备产生中断请求时，会在中断引脚上产生电信号，CPU 会周期性地检查这些引脚的状态以检测中断。

第六章

1. 流水线方式下，一条指令的执行时间缩短了吗？程序的执行时间缩短了吗

每条指令的执行时间可以被缩短，因为多条指令可以并行执行，而不必等待前一条指令完全执行完毕。

然而，流水线方式并不直接影响程序的执行时间，因为程序的执行时间取决于执行所有指令所需的时间。流水线可以加快每条指令的执行，但程序的总体执行时间还是由指令的数量和依赖关系决定。如果程序中存在数据依赖或控制依赖，流水线可能需要等待相关指令完成才能继续执行，从而不能充分发挥流水线的优势。

2. 具有什么特征的指令集易于实现指令流水线？

- 1、指令长度相等:指令长度相等可以使程序计数器(PC)的增量固定, 简化流水线控制逻辑。
- 2、可寻址操作数:指令的操作数在寄存器或内存中均可寻址, 使得内部数据通路的设计更加灵活, 可避免数据相关的冒险和停顿。
- 3、简单指令格式:指令格式简单可以降低解码器的复杂度, 减少占用的面积和功耗。
- 4、在数据通路上有单字读写存储器的支持:这可以减少内存交互和外部缓存的需要, 并且简化了流水线控制逻辑。
- 5、单一寄存器组:单一寄存器组可以简化操作数的传递, 降低数据通路的复杂度。
- 6、避免数据相关:使用一些可预测的执行方式, 例如延迟槽(branch delay slot)和分支预测等, 可以降低数据相关情况的发生, 提高流水线效率。

3. 流水线处理器中时钟周期如何确定? 单发射流水线处理器的CPI为多少? 每个时钟周期一定有一条指令完成吗? 为什么?

流水线处理器的时钟周期通常由流水线中执行时间最长的阶段决定。

单发射流水线处理器的 CPI 为 1。

每个时钟周期不一定有一条指令完成。原因在于: 流水线可能会因为各种情况出现停顿, 比如数据相关、控制相关等。当出现这些相关时, 流水线可能需要插入气泡(空操作)来解决冲突, 导致在某些时钟周期内没有有效指令完成。

4. 流水线处理器的控制器实现方式更类似于单周期控制器还是多周期控制器? 为什么?

流水线处理器的控制器实现方式更类似于**多周期控制器**。

理由如下:

多周期控制器将指令执行分解为多个阶段, 不同阶段使用不同的部件, 部件可以在不同阶段被**重复使用**。而流水线处理器也是将指令执行分成多个流水段, 在不同的流水段中, 部件会被重复使用, 以提高处理器的效率。

单周期控制器中, 每个指令在一个固定长度的时钟周期内完成, 部件不能被重复使用。这与流水线处理器中部件可重复使用的特点不同。

5. 为什么要在各流水段之间加寄存器? 各流水段寄存器的宽度是否都一样? 为什么?

在各流水段之间加寄存器主要是为了实现以下几个目的：

- 隔离不同的流水段，使得各段的操作能够独立进行，互不干扰。
- 保存当前流水段的处理结果，以便下一个流水段能够及时获取并进行后续处理。
- 解决流水段之间的速度不匹配问题，确保数据能够稳定地在流水段之间传递。

各流水段寄存器的宽度不一定都一样。

原因在于：不同流水段处理的数据类型和信息量可能不同。

例如，某些流水段可能处理的是地址信息，其所需的寄存器宽度可能较小；而处理数据运算的流水段可能需要更宽的寄存器来存储数据。寄存器的宽度应根据具体流水段处理的数据特征来确定，以满足数据存储和传递的需求。

6. 你能列出哪几种流水线被阻塞的情况？

流水线受阻一般有三种情况:数据相关、控制相关、结构相关

- ①在指令重叠执行过程中,硬件资源满足不了指令重叠执行要求,发生资源冲突.如在同一时间,几条重叠执行的指令分别要取指令、取操作和存结果,都需要访存,就会发生访存冲突
- ②在程序的相邻指令之间出现了某种关联，如当一条指令需要用到当前指令的执行结果,而这些指令均在流水线中重叠执行,就可能引起数据相关
- ③当流水线遇到分支指令时,如一条指令要等前一条指令（或几条）指令作出转移方向的决定后,才能进入流水线时,便发生控制相关

7. 为什么说Pentium4是“CISC壳、RISC核”结构？

- Pentium 4 指令集在外观上保持了 CISC 的形式：它继承了传统 CISC 指令集丰富多样的指令格式和复杂的指令功能。
- 但其内部核心的微操作采用了 RISC 风格：在处理器内部，将复杂的 CISC 指令分解为一系列简单的、类似于 RISC 的微操作来执行。
- 这种结构结合了 CISC 指令集的兼容性和易用性，以及 RISC 微操作的高效执行特性。

第八章

1. 什么是I/O接口？I/O接口的基本功能有哪些？

I/O接口是一组电路，包含能被CPU直接访问的寄存器或硬件电路，其基本功能包括地址译码、数据缓冲、信息转换、提供命令译码和状态信息以及定时和控制。

2. 串行接口和并行接口的特点各是什么？

串行接口：串行传送方式

优点:传输线少,连线简单,成本低

缺点:速度慢。

适用于远距离或数据量少的通信。

并行接口：并行传送方式

优点:速度快

缺点:传输线多。

适用于近距离大数据通信。

3. CPU如何进行设备的寻址？I/O端口的编址方式有哪两种？各有何特点？

内存映射 I/O 寻址、独立 I/O 寻址、直接寻址、间接寻址

I/O端口与存储器统一编址方式和I/O端口独立编址方式。

I/O端口与存储器统一编址方式:不必专门设置输入/输出指令来访问端口,但是统一编址会减少主存容量。

I/O端口独立编址方式:端口不占主存地址空间,但必须设置专门的输入/输出指令访问端口。

4. 什么是程序查询I/O方式？说明其工作原理。

程序查询 I/O 方式是一种用于与外部设备进行数据交互的输入/输出操作方法，是一种早期的 I/O 控制方式。

工作原理：

- CPU 不断地查询 I/O 设备的状态。
- 当 I/O 设备准备好与 CPU 交换数据时，CPU 才执行数据传输操作。
- 否则，CPU 一直处于等待状态。

例如，假设要从打印机输出数据，CPU 会不断检查打印机是否空闲，如果空闲则将数据发送给打印机进行打印。

5. 什么是中断控制I/O方式？说明其工作原理。

中断控制 I/O 方式是一种常见的 I/O 控制方式，它通过中断信号来实现设备和CPU之间的通信。

工作原理：

- 当 I/O 设备准备好与 CPU 交换数据时，向 CPU 发出中断请求。
- CPU 在执行完当前指令后，响应中断请求，暂停当前程序的执行。
- 转去执行中断服务程序，完成与 I/O 设备的数据交换。
- 中断服务程序执行完毕后，CPU 恢复原来程序的执行。

例如，在从磁盘读取数据时，磁盘准备好数据后发出中断，CPU 响应中断并处理数据读取。

6. 什么是向量中断？说明在向量中断方式下形成中断向量的基本方法。

向量中断：向量中断是指当 CPU 响应中断时，由硬件直接产生一个固定的地址（即中断向量），该地址指向中断服务程序的入口地址。

在向量中断方式下形成中断向量的基本方法：

- 硬件向量法：通过中断向量地址形成部件，根据中断源的编码直接产生中断向量地址。
- 软件查询法：在中断响应时，通过执行一段查询程序来确定中断源，并计算出中断向量地址。

7. 为什么中断控制器把中断请求设备标识（中断类型）放在总线的数据线上，而不是放在地址线上？

地址总线是单向的，只能用于CPU向存储器和外设传输地址信息，而不能用于外设向CPU传输信息，所以外设向CPU传输中断向量只能通过数据总线。

8. 在多周期处理器中并不是每个时钟周期结束都允许响应中断，为什么？如果在一条指令执行过程中，CPU响应中断而转去执行中断服务程序，会产生什么问题？

因为不同指令的执行阶段和时长不同，如果在不恰当的时钟周期响应中断，可能会导致指令执行不完整、数据错误或状态混乱。

可能会产生以下问题：指令执行结果的不确定性、数据的不一致性、程序流程的混乱以及可能导致系统错误或崩溃。

9. 什么是可屏蔽中断？什么是不可屏蔽中断？

可屏蔽中断是通过CPU的INTR引脚引入，当中断允许标志IF为1时，允许中断，IF为0时，中断受到禁止。不可屏蔽中断，不受IF标志的控制由NMI引脚引入的中断。

10. 为什么在保护现场和恢复现场的过程中，CPU必须关中断？

数据一致性:当处理异常或中断时，CPU需要保存当前程序的执行状态、寄存器的内容以及其他关键信息到栈或其他地方。如果不关闭中断，其他中断可能会在保存现场的同时修改这些信息，导致不一致性。

防止嵌套中断:关闭中断可以防止嵌套中断的发生，确保异常或中断处理的顺序性。如果不关闭中断，可能会在处理一个中断时被另一个中断中断。导致混乱。

确保原子性:关闭中断可以确保保护现场和恢复现场的过程是原子性的，即不会被其他中断或线程打断，从而保证了这两个过程的完整性。

因此，关闭CPU中断是为了确保在保护现场和恢复现场的过程中不会出现数据不一致或混乱的情况，以确保程序的正确性和可靠性。

11. DMA方式能够提高成批数据交换效率的主要原因何在？

它能让数据在内存和 I/O 设备之间直接传输，而无需 CPU 过多干预，从而减少了 CPU 等待和处理数据传输的时间，使 CPU 能更专注于其他任务。

12. CPU响应DMA请求和响应中断请求有什么区别？

对中断请求的响应时间只能发生在每条指令执行完毕时,而对 DMA请求的响应时间可以发生在每个机器周期结束时。

13. 在DMA接口中，什么时候给出“DMA请求”（或“总线请求”）信号？什么时候给出“中断请求”信号？

CPU在什么时候响应DMA请求？在什么时候响应中断请求？

在 DMA 接口中，当 I/O 设备准备好进行数据传输时，给出“DMA 请求”（或“总线请求”）信号。当数据传输完成后，给出“中断请求”信号。

CPU 通常在当前指令执行结束后响应 DMA 请求。而 CPU 会在执行完当前指令后，响应中断请求。